AI Engineering Boot Camp

Day 6 — NumPy & ML Foundations

Arrays · Vectorisation · Broadcasting · ML Functions from Scratch

---

Target: Google · Meta · OpenAI · Anthropic

Files Created Today

- day6_numpy_basics.py — arrays, shapes, dtypes, indexing
- day6_vectorisation.py — vectorisation vs loops, matrix multiply
- day6_broadcasting.py — broadcasting, axis, normalisation
- day6_ml_functions.py — softmax, ReLU, MSE, linear layer from scratch

All files pass mypy --strict. All pushed to GitHub.

Why NumPy?

Python lists are slow for math — each element is a separate Python object stored randomly in memory. NumPy arrays store all elements as raw bytes in one continuous block, allowing the CPU to operate on all of them simultaneously.

```
# Python list — slow (visits each element one by one)
numbers = [1, 2, 3, 4, 5]
doubled = [x * 2 for x in numbers]  # Python loop
```

```
# NumPy — fast (operates on entire array at once)
import numpy as np
numbers = np.array([1, 2, 3, 4, 5])
doubled = numbers * 2  # no loop — vectorised
```

Benchmark: 1 million elements — Python loop: 0.20s, NumPy: 0.003s. 70x faster.

A neural network with 1 billion parameters is just a massive array. Every forward pass, every gradient, every weight update — all array operations. NumPy makes this feasible.

NumPy Basics

Creating arrays

```
a = np.array([1, 2, 3, 4, 5])      # from list
b = np.zeros((3, 3))               # 3x3 matrix of zeros
c = np.ones((2, 4))                # 2x4 matrix of ones
```

```
d = np.arange(0, 10, 2)             # [0, 2, 4, 6, 8]
e = np.linspace(0, 1, 5)            # [0, 0.25, 0.5, 0.75, 1.0]
f = np.random.rand(3, 4)            # random floats 0-1, shape (3,4)
```

Shape, dtype, ndim

```
a.shape   # (5,) — dimensions as tuple
a.dtype   # int64 — data type
a.ndim    # 1 — number of dimensions
```

Shape is everything in ML. Always check shapes when debugging.

Reshaping

```
matrix = np.arange(12).reshape(3, 4)  # shape (3, 4)
# [[0,1,2,3], [4,5,6,7], [8,9,10,11]]

# -1 means 'figure it out'
flat = matrix.reshape(-1)       # shape (12,)
col  = matrix.reshape(-1, 1)    # shape (12, 1)
```

Indexing and slicing

```
matrix[0]       # first row: [0, 1, 2, 3]
matrix[1, 2]    # row 1, col 2: value 6
matrix[:, 0]    # ALL rows, col 0: [0, 4, 8]  (first column)
matrix[0, :]    # row 0, ALL cols: [0, 1, 2, 3]  (first row)
matrix[0:2, 1:3]  # rows 0-1, cols 1-2: [[1,2],[5,6]]
```

The colon : means 'everything' in that dimension. matrix[:, 0] = all rows, column 0 = first column.

Vectorisation

Applying an operation to an entire array at once without Python loops. 10-100x faster than loops.

Element-wise operations

```
x = np.array([1.0, 2.0, 3.0, 4.0, 5.0])
x * 2          # [2, 4, 6, 8, 10]
```

```
x ** 2          # [1, 4, 9, 16, 25]
np.sqrt(x)      # [1.0, 1.414, 1.732, 2.0, 2.236]
np.sum(x)       # 15.0
np.mean(x)      # 3.0
np.std(x)       # 1.414
```

Matrix operations

```
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])
```

```
A @ B    # matrix multiply — [[19,22],[43,50]]
A * B    # element-wise    — [[5,12],[21,32]]
```

Operation	Meaning	Example
A @ B	Matrix multiply — dot product of rows and columns	`(2,3) @ (3,4) = (2,4)`
A * B	Element-wise — matching positions multiplied	`[1,2]*[3,4] = [3,8]`

Critical: A @ B and A * B produce completely different results. Using * when you meant @ is a silent bug.

Matrix multiply shape rule

```
(5, 3) @ (3, 4) = (5, 4)
#  ^--- inner dims must match ---^
#  outer dims become result shape
```

In ML: (batch_size, features) @ (features, outputs) = (batch_size, outputs). This is the core of every linear layer.

Broadcasting

NumPy automatically stretches smaller arrays to match larger ones when shapes are compatible. No loops needed.

Scalar broadcasting

```
a = np.array([1, 2, 3, 4, 5])
a + 10   # [11, 12, 13, 14, 15] — 10 broadcast to all elements
a * 3    # [3, 6, 9, 12, 15]
```

Vector broadcast across matrix

```
matrix = np.ones((3, 4))          # shape (3, 4)
vector = np.array([1, 2, 3, 4])   # shape (4,)
matrix + vector                   # vector added to every row
# [[2,3,4,5], [2,3,4,5], [2,3,4,5]]
```

Normalisation — the most important broadcast pattern

```
X = np.random.rand(1000, 128)  # 1000 samples, 128 features
mean = X.mean(axis=0)          # shape (128,) — avg of each feature
std  = X.std(axis=0)           # shape (128,)
X_norm = (X - mean) / std      # broadcast across all 1000 samples
```

Without broadcasting this would require a loop over 1000 samples. Broadcasting does it in one line.

axis parameter

axis	Operates	Result
axis=0	Down the rows (collapses rows)	`One value per column — shape (cols,)`
axis=1	Across columns (collapses cols)	`One value per row — shape (rows,)`
no axis	Entire array	`Single scalar value`

Memory trick: axis=0 crushes the matrix vertically (columns survive). axis=1 crushes horizontally (rows survive).

ML Functions from Scratch

These four functions run inside every neural network on every forward pass. Implement them from memory.

Softmax

Converts a vector of scores into probabilities that sum to 1. Used in the last layer of classification models.

```
def softmax(x: np.ndarray) -> np.ndarray:
    exp_x = np.exp(x - np.max(x))  # subtract max — numerical stability
    return exp_x / np.sum(exp_x)   # type: ignore[no-any-return]
```

Why subtract np.max(x)? np.exp() of large numbers overflows to inf. Subtracting the max keeps values small without changing the final probabilities. This is called numerical stability.

```
softmax([2.0, 1.0, 0.5]) → [0.629, 0.231, 0.140]
sum of output always = 1.0
```

ReLU — Rectified Linear Unit

Most common activation function. Zeros out negatives, keeps positives unchanged.

```
def relu(x: np.ndarray) -> np.ndarray:
    return np.maximum(0, x)  # type: ignore[no-any-return]
```

```
relu([-3, -1, 0, 2, 4]) → [0, 0, 0, 2, 4]
```

Mean Squared Error

Measures how wrong predictions are. Common loss function for regression.

```
def mse(predictions: np.ndarray, targets: np.ndarray) -> float:
    return float(np.mean((predictions - targets) ** 2))
```

```
mse([2.5, 0.5, 2.0, 8.0], [3.0, -0.5, 2.0, 7.0]) → 0.5625
```

Linear Layer

The core of every neural network layer. Input matrix × weight matrix + bias.

```
def linear(X: np.ndarray, W: np.ndarray, b: np.ndarray) -> np.ndarray:
    return X @ W + b  # type: ignore[no-any-return]
```

```
X: shape (5, 3)   — 5 samples, 3 input features
W: shape (3, 4)   — weights: 3 inputs → 4 outputs
b: shape (4,)     — bias for each output
output: shape (5, 4)  — 5 samples, 4 output features
```

Shape rule: (5,3) @ (3,4) = (5,4). Inner dimensions must match. Outer dimensions become the result.

Anki Cards — Day 6

- Q: What is vectorisation? A: Applying operation to whole array at once — no Python loops
- Q: What does axis=0 mean? A: Operates down the rows — result is one per column
- Q: What does axis=1 mean? A: Operates across columns — result is one per row
- Q: What is broadcasting? A: NumPy stretches smaller arrays to match larger ones
- Q: Why subtract max before softmax? A: Numerical stability — prevents exp() overflow to inf
- Q: What does A @ B do vs A * B? A: @ is matrix multiply, * is element-wise multiply
- Q: What is the shape rule for matrix multiply? A: (a,b) @ (b,c) = (a,c) — inner dims must match
- Q: What does np.arange(0,10,2) produce? A: [0, 2, 4, 6, 8]
- Q: What does matrix[:, 0] return? A: All rows, column 0 — the entire first column
- Q: What does ReLU do? A: Zeros out negatives, keeps positives — np.maximum(0, x)

Quick Reference — NumPy Cheatsheet

Array creation

```
np.array([1,2,3])          # from list
np.zeros((3,3))            # zeros
np.ones((2,4))             # ones
np.arange(0,10,2)          # [0,2,4,6,8]
np.linspace(0,1,5)         # 5 evenly spaced from 0 to 1
np.random.rand(3,4)        # random uniform 0-1
np.random.randn(3,4)       # random normal (mean=0, std=1)
```

Array info

```
a.shape    # dimensions tuple
a.dtype    # data type
a.ndim     # number of dimensions
a.size     # total number of elements
```

Math operations

```
np.sum(a)               # sum all elements
np.sum(a, axis=0)       # sum down rows
np.mean(a)              # mean
np.std(a)               # standard deviation
np.max(a)               # maximum
np.min(a)               # minimum
np.sqrt(a)              # square root
np.exp(a)               # e^x
np.log(a)               # natural log
A @ B                   # matrix multiply
A * B                   # element-wise multiply
```

Reshaping

```
a.reshape(3,4)          # new shape
a.reshape(-1)           # flatten to 1D
a.T                     # transpose
```

Tomorrow — Day 7: Exit test + first real project. Build a full data pipeline using everything from the week.